Faculty of Computers and Artificial Intelligence
Cairo University

CS213:ProgrammingII(Object OrientedProgramming)

Year2026-2027, First Semester

2026 / 10 / 24

Assignment 2 - Part 1

Task 1: OOP Design of the Vole Machine Simulator

Course instructor: Dr. Mohammad El-Ramly

Team members

Name: Reyan Saeed Ali Omran | **ID**: 20253071 | **Email**: 20253071@stud.fci-cu.edu.eg | **Section**: S3,4

Name:BasilBadr | **ID**: 20240110 | **Email**: 20240110@stud.fci-cu.edu.eg | **Section**:S3,S4

Name: Salah Alden Amer Qarmout | **ID**: 20220806 | **Email**: 20220806@stud.fci-cu.edu.eg | **Section**: S5,6

Name: Antonuos Abdo Shaker | **ID**: 20251058 | **Email**: 20251058@stud.fci-cu.edu.eg | **Section**: S3,4

Team leader: Reyan | Phone: +201118134560

Work breakdown

Reyan: Recorded the UML class diagrams explanation video.

Basil: Prepared the file,and made the UML class diagrams initial design.

Salah Alden: Combined the best part of the three AI designs into the final design.

Antonuos: Gave the same prompt to the three AI's,and collected there responds to the UML classes digrams

1. Introduction

TheVole machine isafictitious, very small computer used to teach how a computer executes programs. The simulator loads a program into a simulated memory and executes it instruction by instruction.

Architecture

Memory: 256 cells, addresses 00 to FF, each cell 8 bits (2 hex digits).
Registers: 16 general purpose registers, numbered 0 to F, each 8 bits.
Program Counter (PC): Holds the address of the next instruction.
Instruction Register (IR): Holds the current instruction (16 bits = 4 hex digits).
Instruction format: 16 bits in two consecutive memory cells: first hex digit is the op-code, the other three are the operand (R, S, T or X, Y).
Screen: Memory cell FF is mapped to the screen: storing a value in FF (3RFF) prints it.
Program startaddress: Must be an even address below FF.

Instruction set

1 RXY: Load register R from memory cell XY
2 RXY: Load register R with the value XY
3 RXY: Store register R into memory cell XY
3 RFF: Store register R to the screen (cell FF)
4 0RS: Move register R to register S
5 RST: R = S + T (two's complement)
6 RST: R = S + T (floating point)
7 RST: R = S OR T
8 RST: R = S AND T
9 RST: R = S XOR T
A R0X: Rotate register R right X times (circular)
B RXY: If register R == register 0, jump to address XY (PC = XY)
C 000: Halt

2. Initial Design

2.1 Classes and responsibilities

Simulator: User interface: shows the menu,readschoices and calls the other classes. No machine logic.

FileLoader: Reads a program file, ignores comment (;) and empty lines, keeps the first 4 hex digits of each line, validates them and writes them into memory from an even start address.

Machine: Owns the CPU, Memory and Screen. Starts the program, runs it all at once or step by step and displays the machine state.

CPU: Owns PC, IR and the registers. Implements the fetch, decode, execute cycle and updates PC.

Memory: Stores the 256 cells with validated read/write and a 16x16 matrix display.

Registers: Stores the 16 general purpose registers (get, set, display).

Instruction: Represents one decoded instruction (op-code and operand fields) and checks that it is valid.

ALU: Integer add, floating-point add, OR, AND, XOR and rotate.

Screen:Receives values written to addressFF and displays them.

2.2 Relations between classes

Simulator -> Machine (Association): Thesimulator uses one machine.

Simulator -> FileLoader (Association): The simulator asks the loader to read program files.

FileLoader ..> Memory (Dependency): Writes the program into memory, does not own it.

Machine *-- CPU, Memory, Screen (Composition): The machine is made of them; they do not exist without it.

CPU *-- Registers, ALU (Composition): Registers and ALU are internal parts of the CPU.

CPU ..> Instruction (Dependency): Creates an Instruction at each decode step.

CPU -> Memory (Association): Reads instructions and data, writes data.

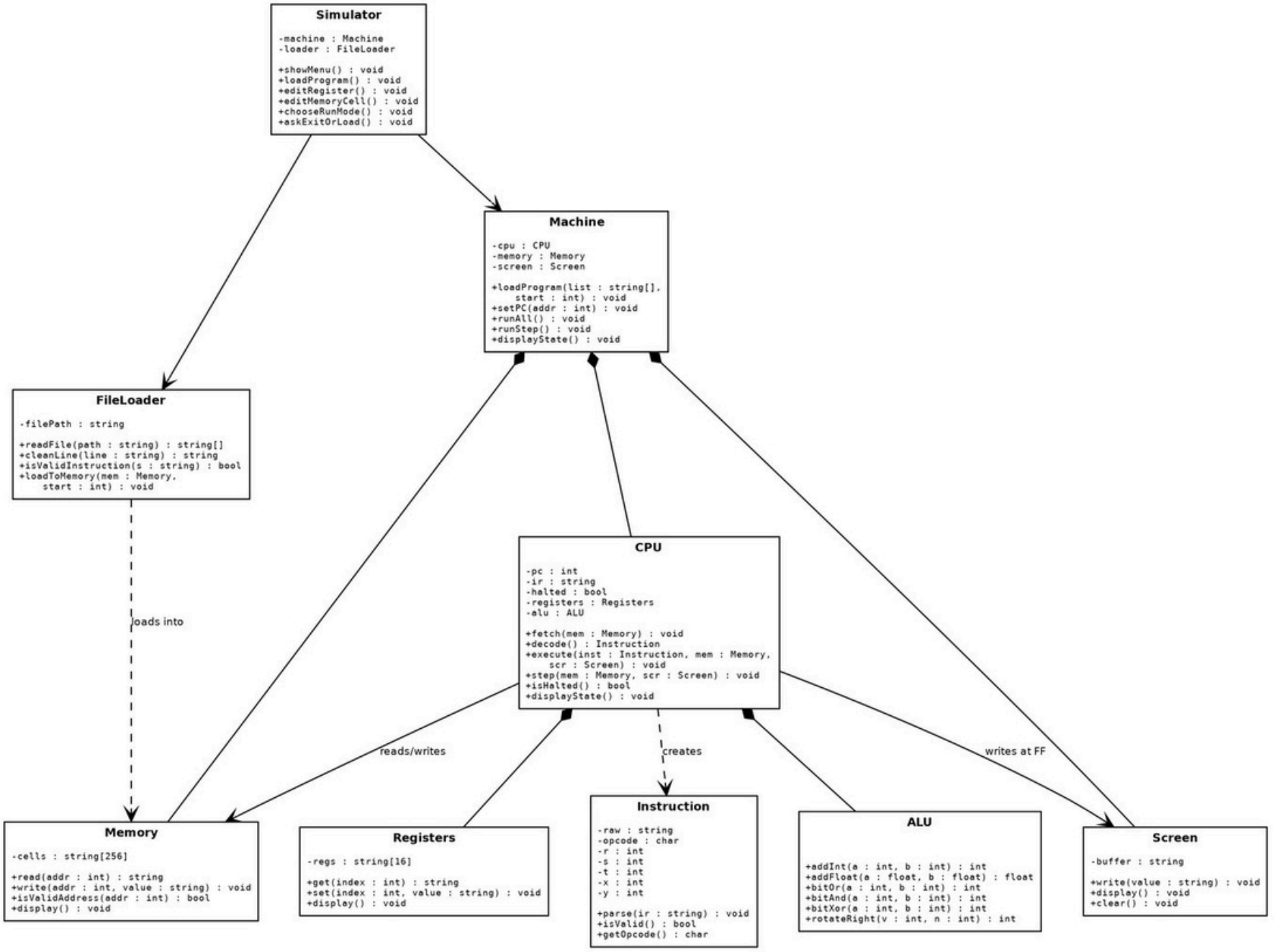
CPU -> Screen (Association): Writes to the screen when the address is FF.

2.3 PlantUML code

```
@startuml
skinparam classAttributeIconSize 0
class Simulator {
 -machine : Machine
 -loader : FileLoader
 +showMenu() : void
 +loadProgram() : void
 +editRegister() : void
 +editMemoryCell() : void
 +chooseRunMode() : void
 +askExitOrLoad() : void
}
class FileLoader {
 -filePath : string
 +readFile(path : string) : string[]
 +cleanLine(line : string) : string
 +isValidInstruction(s : string) : bool
 +loadToMemory(mem : Memory, start : int) : void
}
class Machine {
 -cpu : CPU
 -memory : Memory
 -screen : Screen
 +loadProgram(list : string[], start : int) : void
 +setPC(addr : int) : void
 +runAll() : void
 +runStep() : void
 +displayState() : void
}
class CPU {
 -pc : int
 -ir : string
 -halted : bool
 -registers : Registers
 -alu : ALU
 +fetch(mem : Memory) : void
 +decode() : Instruction
  +execute(inst : Instruction, mem : Memory, scr : Screen) : void
+step(mem : Memory, scr : Screen) : void
 +isHalted() : bool
 +displayState() : void
}
```

```
class Memory {
 -cells : string[256]
 +read(addr : int) : string
 +write(addr : int, value : string) : void
+isValidAddress(addr : int) : bool
+display() : void
}
class Registers {
 -regs : string[16]
 +get(index : int) : string
 +set(index : int, value : string) : void
+display() : void
}
class Instruction {
 -raw : string
 -opcode : char
 -r : int
 -s : int
 -t : int
 -x : int
 -y : int
 +parse(ir : string) : void
 +isValid() : bool
 +getOpcode() : char
}
class ALU {
+addInt(a : int, b : int) : int
+addFloat(a : float, b : float) : float
+bitOr(a : int, b : int) : int
 +bitAnd(a : int, b : int) : int
 +bitXor(a : int, b : int) : int
+rotateRight(v : int, n : int) : int
}
class Screen {
 -buffer : string
 +write(value : string) : void
+display() : void
 +clear() : void
}
Simulator --> Machine
Simulator --> FileLoader
FileLoader ..> Memory : loads into
Machine *-- CPU
Machine *-- Memory
Machine *-- Screen
CPU *-- Registers
CPU *-- ALU
CPU ..> Instruction : creates
CPU --> Memory : reads/writes
CPU --> Screen : writes at FF
@enduml
```

2.4 UML class diagram



3. AI Designs

Prompt(the same prompt was given to the three AIs)

am a student. I need an object-oriented design (UML class diagram, written as PlantUML code) for a simulator of the Vole machine. No implementation code, only classes, attributes, methods and relations. Please also explain the responsibility of each class and why you designed it this way.

THE VOLE MACHINE
- Memory: 256 cells (addresses 00 to FF), each cell is 8 bits (2 hex digits).
- 16 general purpose registers (0 to F), each 8 bits.
- Program Counter (PC): address of the next instruction.
- Instruction Register (IR): holds the current instruction.
- Each instruction is 16 bits = 4 hex digits = 2 memory cells.
- Address FF is memory-mapped to the screen: writing to FF prints on screen.

INSTRUCTION SET (op-code, operand, meaning)

1 RXYload register R from memory cell XY
2 RXYload register R with the value XY
3 RXYstore register R into memory cell XY
3 RFFstore register R to the screen
4 0RSmove register R to register S
5 RSTR = S + T (two's complement)
6 RSTR = S + T (floating point)
7 RSTR = S OR T
8 RSTR = S AND T
9 RSTR = S XOR T
A R0Xrotate register R right X times (circular)
B RXYif register R == register 0, jump to address XY (PC = XY)
C 000   halt

THE SIMULATOR MUST
1. Offer a menu of choices.
2. Load a program from a file into a chosen memory address that must be even and less than FF.
3. Let the user overwrite any register or memory cell with a new value.
4. Set the PC to the start address of the program.
5. Display memory as a matrix, plus IR and the 16 registers.
6. Fetch each instruction into IR, validate it, decode it, execute it and update the PC.
7. Let the user run the program all at once or step by step.
8. If all at once: display memory, PC, IR, registers and the screen after the program finishes.
9. If step by step: display them after each step, then wait for Enter for the next step.
10. Finally ask the user to exit or load another program.

PROGRAM FILE FORMAT
A text file with one instruction per line (4 hex digits as a string). Lines starting with ; are comments and are ignored. Empty lines are ignored. Anything after the 4 hex digits on a line is ignored.

OUTPUT REQUIRED
PlantUML code for the class diagram (classes with attributes and methods, and relations such as association, composition, dependency), plus a short explanation of each class.

3.1 AI number 1: ChatGPT

chatgpt:

The design below uses object-oriented principles, separation of responsibilities, and composition to model the Vole machine simulator. It includes the required menu, program loading, memory, registers, instruction execution, screen output, validation, and step-by-step or continuous execution.
The main design decision is to separate the machine hardware, the instruction execution logic, and the user interface. This makes the design easier to understand, test, and extend without putting everything into one large class.
1. PlantUML class diagram
Copy the following code into a PlantUML editor to generate the UML class diagram.
Machine components
Class Responsibility
VoleMachine Represents the complete machine. Owns the memory, register file, CPU, and screen, and provides controlled access to them.
Memory Stores 256 eight-bit cells with addresses 00–FF. Validates addresses and supports reading, writing, and loading bytes.
RegisterFile Manages the 16 general-purpose registers, numbered 0–F. Validates register indices and stores eight-bit values.
CPU Maintains the Program Counter (PC), Instruction Register (IR), and halted state. Fetches instructions and coordinates decoding and execution.
Screen Collects output produced by writes to memory address FF and provides the accumulated output for display.

PlantUML code

```
@startuml
titleVoleMachineSimulator - Class Diagram

 skinparam classAttributeIconSize 0
 skinparam linetype ortho



 class SimulatorApp {
  - ui: ConsoleUI
  - machine: VoleMachine
  - loader: ProgramLoader
  - controller: ExecutionController
 + start(): void
  + loadAnotherProgram(): void
  + exit(): void
 }


class ConsoleUI {
 + showMenu(): int
 + getProgramFilePath(): String
 + getStartAddress(): int
 + getExecutionMode(): ExecutionMode
 + getRegisterInput(): RegisterInput
 + getMemoryInput(): MemoryInput
 + displayMachineState(machine: VoleMachine): void +
displayMemoryMatrix(memory: Memory): void
 + displayRegisters(registers: RegisterFile): void +
displayScreen(screen: Screen): void
 + waitForEnter(): void
 + showError(message: String): void
}
```

```
class VoleMachine {
 - memory: Memory
 - registers: RegisterFile
 - cpu: CPU
 - screen: Screen
 + readMemory(address: int): Byte
 + writeMemory(address: int, value: Byte): void
+ readRegister(index: int): Byte
 + writeRegister(index: int, value: Byte): void
+ setPC(address: int): void
 + reset(): void
}


 class Memory {
  - cells: Byte[256]
  + read(address: int): Byte
  + write(address: int, value: Byte): void
  + loadBytes(startAddress: int, data: Byte[]): void
 + getCells(): Byte[]
  + validateAddress(address: int): boolean
 }



 class RegisterFile {
  - registers: Byte[16]
  + read(index: int): Byte
  + write(index: int, value: Byte): void
 + getAll(): Byte[]
  + validateIndex(index: int): boolean
 }


class CPU {
 - pc: int
 - ir: Instruction
 - halted: boolean
 + fetch(machine: VoleMachine): String +
step(machine: VoleMachine): void
 + setPC(address: int): void
 + getPC(): int
 + getIR(): Instruction
 + isHalted(): boolean
```

```
 + halt(): void
}

class Instruction {
 - rawCode: String
 - opcode: String
 - digit2: String
 - digit3: String
 - digit4: String
 + getOpcode(): String
 + getRegisterR(): int
 + getRegisterS(): int
 + getRegisterT(): int
 + getAddressXY(): int
 + getImmediateXY(): Byte }



class InstructionDecoder {
 + validate(rawCode: String): boolean
 + decode(rawCode: String): Instruction
 + validateOperands(instruction: Instruction): boolean
}

class InstructionExecutor {
 + execute(instruction: Instruction, machine: VoleMachine): void
 - executeLoad(instruction: Instruction, machine: VoleMachine): void
 - executeStore(instruction: Instruction, machine: VoleMachine): void
 - executeMove(instruction: Instruction, machine: VoleMachine): void
  - executeAddInteger(instruction: Instruction, machine: VoleMachine): void -
executeAddFloat(instruction: Instruction, machine: VoleMachine): void
 - executeLogic(instruction: Instruction, machine: VoleMachine): void
 - executeRotate(instruction: Instruction, machine: VoleMachine): void
 - executeBranch(instruction: Instruction, machine: VoleMachine): void
}


class ProgramLoader {
 + loadProgram(filePath: String, startAddress: int, machine: VoleMachine): void
+ parseFile(filePath: String): Byte[]
 + validateStartAddress(address: int): boolean
 + parseInstruction(line: String): String
}

class ExecutionController {
 - machine: VoleMachine
 - ui: ConsoleUI
 + run(mode: ExecutionMode): void
+ runContinuously(): void
 + runStepByStep(): void
 + executeOneStep(): void
}


class Screen {
 - output: String
 + write(value: Byte): void
+ display(): String
 + clear(): void
}

enum ExecutionMode {
CONTINUOUS
STEP_BY_STEP
}

class RegisterInput {
+ registerIndex: int
+ value: Byte
}

class MemoryInput {
+ address: int
 + value: Byte
}

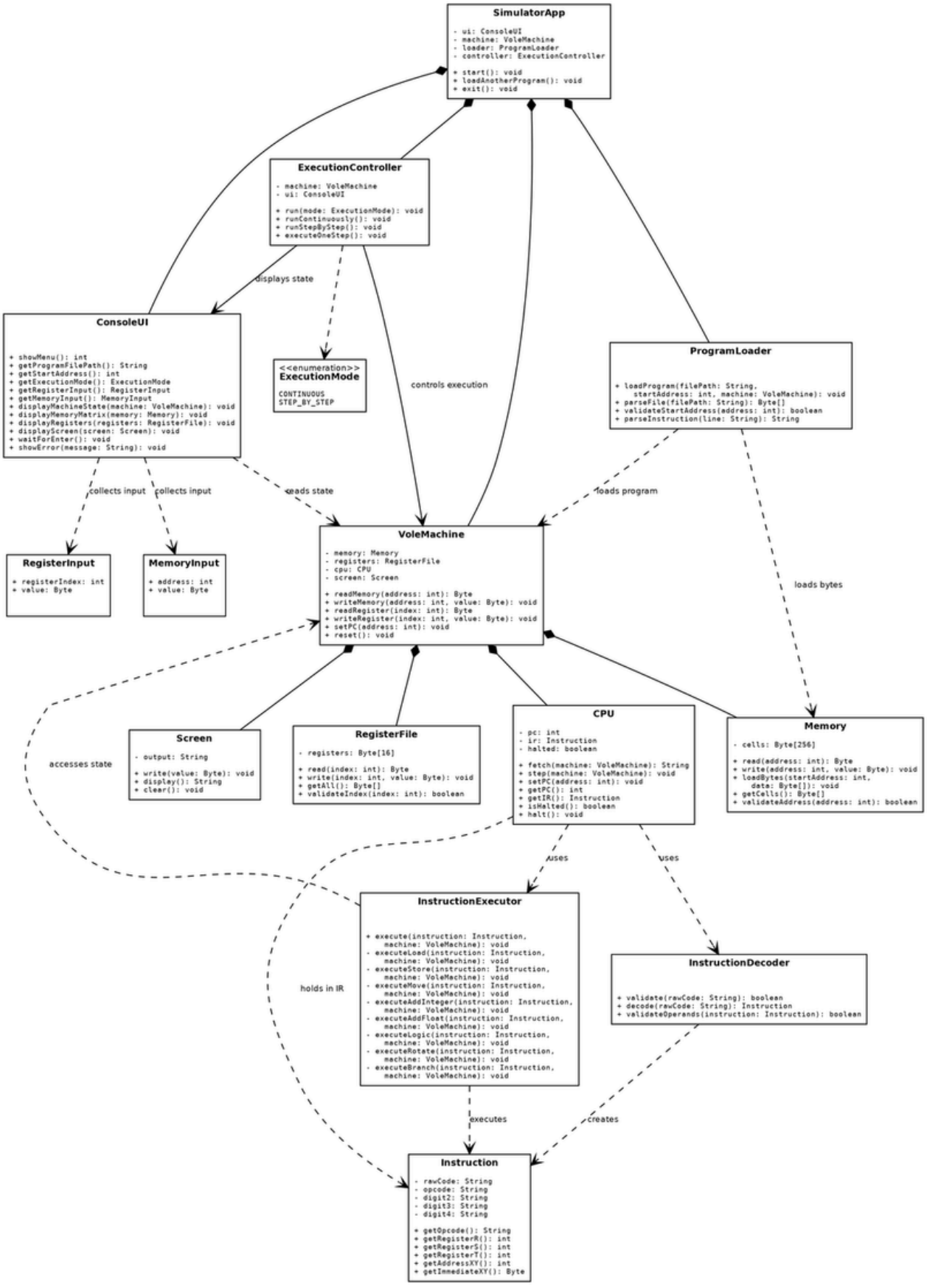
SimulatorApp *-- ConsoleUI
SimulatorApp *-- VoleMachine
SimulatorApp *-- ProgramLoader
SimulatorApp *-- ExecutionController

VoleMachine *-- Memory
VoleMachine *-- RegisterFile
VoleMachine *-- CPU
VoleMachine *-- Screen

CPU ..> InstructionDecoder : uses
CPU ..> InstructionExecutor : uses
CPU ..> Instruction : holds in IR
InstructionDecoder ..> Instruction : creates
InstructionExecutor ..> Instruction : executes
InstructionExecutor ..> VoleMachine : accesses state


ProgramLoader ..> VoleMachine : loads program
ProgramLoader ..> Memory : loads bytes
ExecutionController --> VoleMachine : controls execution
ExecutionController --> ConsoleUI : displays state
ConsoleUI ..> VoleMachine : reads state
ConsoleUI ..> RegisterInput : collects input
ConsoleUI ..> MemoryInput : collects input
ExecutionController ..> ExecutionMode
@enduml
```

UML class diagram - ChatGPT



3.2 number 2: Gemini

GImini:
Here is an object-oriented design for the Vole Machine Simulator using PlantUML, followed by a detailed explanation of each class and the design rationale.

PlantUML Class Diagram Code

You can paste this code into any PlantUML editor (such as PlantText) to generate the visual diagram.
Class Responsibility Explanations & Design Rationale

1. Core Hardware Architecture
Memory

Responsibility: Encapsulates the 256 memory cells (addresses 00 to FF). Handles address bounds checking and memory-mapped I/O for address FF (writing to FF redirects input to screenOutput).

Rationale: Isolates memory storage and screen output logic from execution routines. By handling memory-mapped writing inside Memory.write(), the CPU and instructions don't need special-case logic for screen printing.

RegisterBank

Responsibility: Holds and manages the 16 general-purpose registers (R0–RF).

Rationale: Encapsulates the storage array to prevent out-of-bounds access and ensures safe reading/writing of register data.

CPU

Responsibility: Maintains execution state via the Program Counter (PC) and Instruction Register (IR). Coordinates the Fetch-Decode-Execute cycle.

Rationale: The CPU coordinates execution without needing to know specific operational details of each individual instruction.

Machine

Responsibility: Acts as the primary Facade for the hardware layer. It composes CPU, Memory, and RegisterBank into a single operational machine unit, managing program loading and lifecycle states (e.g., halted vs. running).

Rationale: Provides high-level methods (step(), runAll(), reset()) so the UI controller interacts with one clean interface rather than directly manipulating separate hardware elements.

2. Instruction Set Architecture (Command Pattern)
Instruction (Abstract Class)

Responsibility: Serves as the abstract base class for all Vole machine instructions. Defines the contract method execute(cpu, memory, registers).

Rationale: Implements the Command Pattern. Treating each opcode as a separate class makes the simulator extensible—if new opcodes are added, existing CPU logic remains untouched (Open/Closed Principle).

Concrete Instructions (LoadMemInstruction, JumpInstruction, Add2sCompInstruction, etc.)

Responsibility: Implement the specific arithmetic, bitwise, memory transfer, or control-flow behavior for a single opcode (1 through C).

Rationale: Separating complex arithmetic (like IEEE 754 floating-point addition for Opcode 6 vs. 2's complement for Opcode 5) into dedicated classes keeps code highly modular and simple to test independently.

InstructionDecoder

Responsibility: Reads a 16-bit raw instruction word from IR, validates opcode format, extracts operands (R, X, Y, S, T), and instantiates the matching concrete Instruction subclass.

Rationale: Implements the Factory Pattern. Decouples instruction parsing from instruction execution.

3. Support, File Parsing, and UI Layer

ProgramLoader

Responsibility: Reads external text files, strips comments starting with ; and trailing text, ignores blank lines, validates 4-digit hex strings, and parses them into machine word streams.
Rationale: Isolates file system operations and text parsing from hardware logic.

DisplayUI

Responsibility: Handles formatted console display: printing memory as a 16x16 matrix, displaying register values, showing CPU status, and prompting the user for input.

Rationale: Adheres to the Single Responsibility Principle. Hardware classes hold state; DisplayUI is strictly responsible for rendering state to the user.

SimulatorController

Responsibility: Acts as the application's orchestrator (MVC Controller). Manages menu options, coordinates user edits to registers/memory, manages stepping mode vs. continuous run mode, and prompts for program reloads or exit.

Rationale: Connects user interaction (DisplayUI), file loading (ProgramLoader), and hardware execution (Machine) without tight coupling between UI and core simulator logic.

PlantUML code

```
@startuml
skinparamclassAttributeIconSize 0


class Memory {
    - cells: Byte[256]
    - screenOutput: String
    + Memory()
    + read(address: int): Byte
    + write(address: int, value: Byte): void
    + clear(): void
    + getScreenOutput(): String
}


class RegisterBank {
    - registers: Byte[16]
    + RegisterBank()
    + get(regNum: int): Byte
    + set(regNum: int, value: Byte): void
    + clear(): void
}

class CPU {
    - pc: Byte
    - ir: Word
    + CPU()
    + fetch(memory: Memory): void
    + decodeAndExecute(memory: Memory, registerBank: RegisterBank): bool +
    getPC(): Byte
    + setPC(value: Byte): void
    + getIR(): Word
    + setIR(value: Word): void
    + reset(): void
}


class Machine {
    - cpu: CPU
    - memory: Memory
    - registerBank: RegisterBank
    - isHalted: bool
    + Machine()
    + loadProgram(program: List<Word>, startAddress: int): void
    + step(): bool
    + runAll(): void
    + reset(): void
    + isHalted(): bool
    + getCPU(): CPU
    + getMemory(): Memory
    + getRegisterBank(): RegisterBank
}


abstract class Instruction {
    # opcode: int
    # r: int
    # x: int
    # y: int
    + {abstract} execute(cpu: CPU, memory: Memory, registers: RegisterBank): void
}


class InstructionDecoder <<utility>> {
    + {static} decode(rawInstruction: Word): Instruction
}

class LoadMemInstruction {
    + execute(cpu: CPU, memory: Memory, registers: RegisterBank): void
}
class LoadValInstruction {
    + execute(cpu: CPU, memory: Memory, registers: RegisterBank): void
}
class StoreInstruction {
    + execute(cpu: CPU, memory: Memory, registers: RegisterBank): void
}
class MoveInstruction {
    + execute(cpu: CPU, memory: Memory, registers: RegisterBank): void
}
class Add2sCompInstruction {
    + execute(cpu: CPU, memory: Memory, registers: RegisterBank): void
```

```
}
class AddFloatInstruction {
    + execute(cpu: CPU, memory: Memory, registers: RegisterBank): void
}
class OrInstruction {
    + execute(cpu: CPU, memory: Memory, registers: RegisterBank): void
}
class AndInstruction {
    + execute(cpu: CPU, memory: Memory, registers: RegisterBank): void
}
class XorInstruction {
    + execute(cpu: CPU, memory: Memory, registers: RegisterBank): void
}
class RotateInstruction {
    + execute(cpu: CPU, memory: Memory, registers: RegisterBank): void
}
class JumpInstruction {
    + execute(cpu: CPU, memory: Memory, registers: RegisterBank): void
}
class HaltInstruction {
    + execute(cpu: CPU, memory: Memory, registers: RegisterBank): void
}

             <|--   LoadMemInstruction
             <|--   LoadValInstruction
              <|--    StoreInstruction
               <|--     MoveInstruction
  Instruction <|-- Add2sCompInstruction
   Instruction <|-- AddFloatInstruction
Instruction     <|--      OrInstruction
Instruction    <|--     AndInstruction
Instruction    <|--     XorInstruction
Instruction   <|--    RotateInstruction
Instruction    <|--     JumpInstruction
Instruction <|-- HaltInstruction


class ProgramLoader {
    + loadFromFile(filePath: String): List<Word>
    - parseLine(line: String): String
    - validateHex(hexStr: String): bool
}

class DisplayUI {
    + showMenu(): int
    + showMemoryMatrix(memory: Memory): void
    + showRegisters(registers: RegisterBank): void
    + showCPUState(cpu: CPU): void
    + showFullState(machine: Machine): void
    + promptForAddress(): int
    + promptForByteValue(): Byte
    + promptForRegisterIndex(): int
    + promptForFilePath(): String
    + waitForEnter(): void
}


class SimulatorController {
    - machine: Machine
    - display: DisplayUI
    - loader: ProgramLoader
    + SimulatorController()
    + start(): void
    - handleLoadProgram(): void -
    handleMemoryEdit(): void
    -  handleRegisterEdit():  void
    - handleRunAll(): void
    - handleStepByStep(): void
}

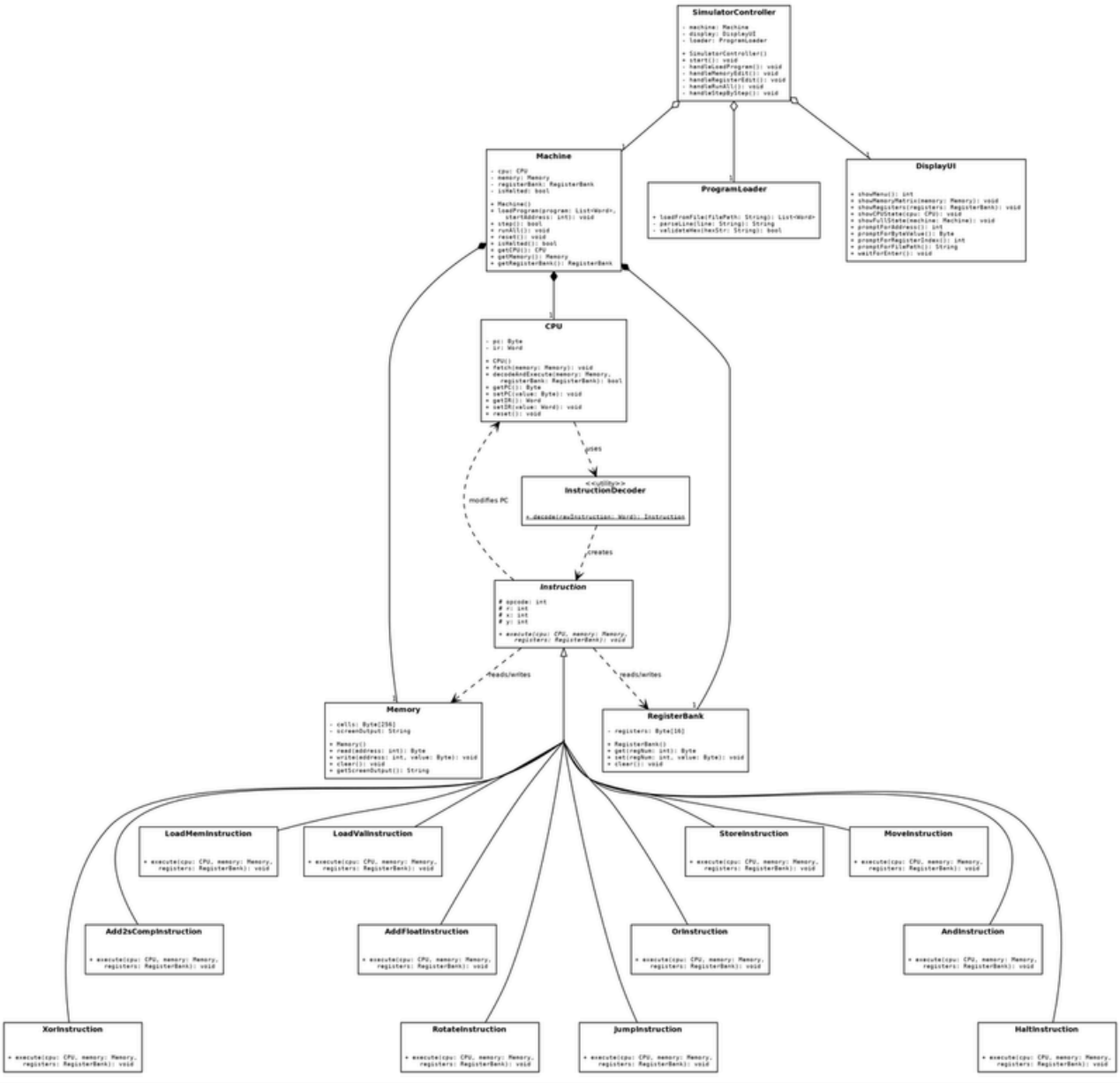

Machine *-- "1" CPU
Machine *-- "1" Memory
Machine *-- "1" RegisterBank

SimulatorController o-- "1" Machine
SimulatorController o-- "1" DisplayUI
SimulatorController o-- "1" ProgramLoader

CPU ..> InstructionDecoder : uses
InstructionDecoder ..> Instruction : creates

Instruction ..> CPU : modifies PC
Instruction ..> Memory : reads/writes
Instruction ..> RegisterBank : reads/writes
@enduml
```

UML class diagram - Gemini



3.3 AI number 3: Claude

Memory: 256 cells;r ead, write, load an instruction.
Register / RegisterFile: One 8-bit register; the collection of 16 registers indexed 0-F.
Screen: Collects everything printed through address FF.
CPU: Holds PC and IR; runs the cycle fetch, decode and validate, execute, update PC in step().
ALU: Two's complement add, floating-point add, OR, AND, XOR, rotate.
Machine: Groups CPU, Memory, RegisterFile, Screen and ALU; reset and user edits of registers and cells.
Instruction: Immutable decoded instruction (opcode, R, S, T, XY).
InstructionDecoder: Splits the raw 16-bit word into fields.
InstructionValidator: Checks the op-code and operands are legal.
Operation (abstract) + 12 subclasses: One class per opcode with execute() (Command pattern).
OperationFactory: Maps an opcode to its Operation, so the CPU needs no switch.
ProgramLoader: Reads the file, skips comments and blank lines, checks the even start address below FF, writes into memory.
ConsoleUI: All console input and output (menu, matrix display, waiting for Enter).
Simulator / RunMode: Controller: menu loop, run all or step by step, start address, exit or reload.
Exceptions: InvalidInstructionException, InvalidProgramFileException.

Why this design

Hardware,d ecoding, validation, execution and UI are in separate classes (single responsibility). Composition models ownership; dependencies show where a class only uses another. One Operation class per opcode keeps the design open for extension.

PlantUML code

```
@startuml VoleSimulator
skinparam classAttributeIconSize 0

class Memory {
 - cells : int[256]
 + read(address : int) : int
 + write(address : int, value : int) : void
 + loadInstruction(address : int, instr : Instruction) : void
+ clear() : void
 + getCells() : int[]
}


class Register {
 - value : int
 + getValue() : int
 + setValue(value : int) : void
}


class RegisterFile {
 - registers : Register[16]
 + get(index : int) : int
 + set(index : int, value : int) : void
+ clear() : void
}


class Screen {
 - buffer : String
 + print(value : int) : void
+ getContent() : String
 + clear() : void
}



class Instruction {
 - opcode : int
 - r : int
 - s : int
 - t : int
 - xy : int
 + getOpcode() : int
 + getR() : int
 + getS() : int
 + getT() : int
 + getXY() : int
 + toHexString() : String }



 class InstructionDecoder {
  + decode(raw : int) : Instruction
 }


class InstructionValidator {
 + isValid(instr : Instruction) : boolean
 + getErrorMessage(instr : Instruction) : String
}


abstract class Operation {
 + {abstract} execute(instr : Instruction, cpu : CPU) : void
}

class LoadFromMemoryOp
class LoadValueOp
```

```
class StoreOp
class  MoveOp
class       AddIntOp
class    AddFloatOp
class  OrOp
class AndOp
class XorOp
class RotateOp       class
JumpIfEqualOp  class
HaltOp


class OperationFactory {
 - operations : Map<int, Operation>
 + getOperation(opcode : int) : Operation
}

class ALU {
 + addTwosComplement(a : int, b : int) : int
 + addFloat(a : int, b : int) : int
 + bitOr(a : int, b : int) : int
 + bitAnd(a : int, b : int) : int
 + bitXor(a : int, b : int) : int
 + rotateRight(value : int, times : int) : int }



class CPU {
 - pc : int
 - ir : Instruction
 - halted : boolean
 + fetch() : void
 + decodeAndValidate() : Instruction +
execute() : void
 + step() : void
 + getPC() : int
 + setPC(address : int) : void
 + getIR() : Instruction
 + isHalted() : boolean
 + halt() : void
 + reset() : void
}


class Machine {
 + reset() : void
 + setRegister(index : int, value : int) : void
 + setMemoryCell(address : int, value : int) : void
}

class ProgramLoader {
 + load(filePath : String, startAddress : int, memory : Memory) : int
- parseLine(line : String) : int
 - isIgnorable(line : String) : boolean
 + validateStartAddress(address : int) : boolean
}

class Simulator {
 - runMode : RunMode
 + start() : void
 + loadProgram() : void
 + editRegisterOrMemory() : void
+ setStartAddress() : void
 + run() : void
 + runAll() : void
 + runStepByStep() : void
 + askExitOrReload() : boolean
}


enum RunMode {
ALL_AT_ONCE
STEP_BY_STEP
}

class ConsoleUI {
 + showMenu() : int
 + readString(prompt : String) : String
 + readInt(prompt : String) : int
 + waitForEnter() : void
 + showMessage(msg : String) : void
 + showState(machine : Machine) : void
 + showMemory(memory : Memory) : void
 + showRegisters(regs : RegisterFile) : void
+ showPC_IR(cpu : CPU) : void
 + showScreen(screen : Screen) : void
}


class InvalidInstructionException
class InvalidProgramFileException

Machine *-- "1" CPU
Machine *-- "1" Memory
Machine   *--   "1"   RegisterFile
Machine *-- "1" Screen
Machine *-- "1" ALU
RegisterFile  *--   "16"  Register
CPU  *--  "1"  InstructionDecoder
CPU  *--  "1"  InstructionValidator
CPU *-- "1" OperationFactory
CPU o-- "0..1" Instruction : ir

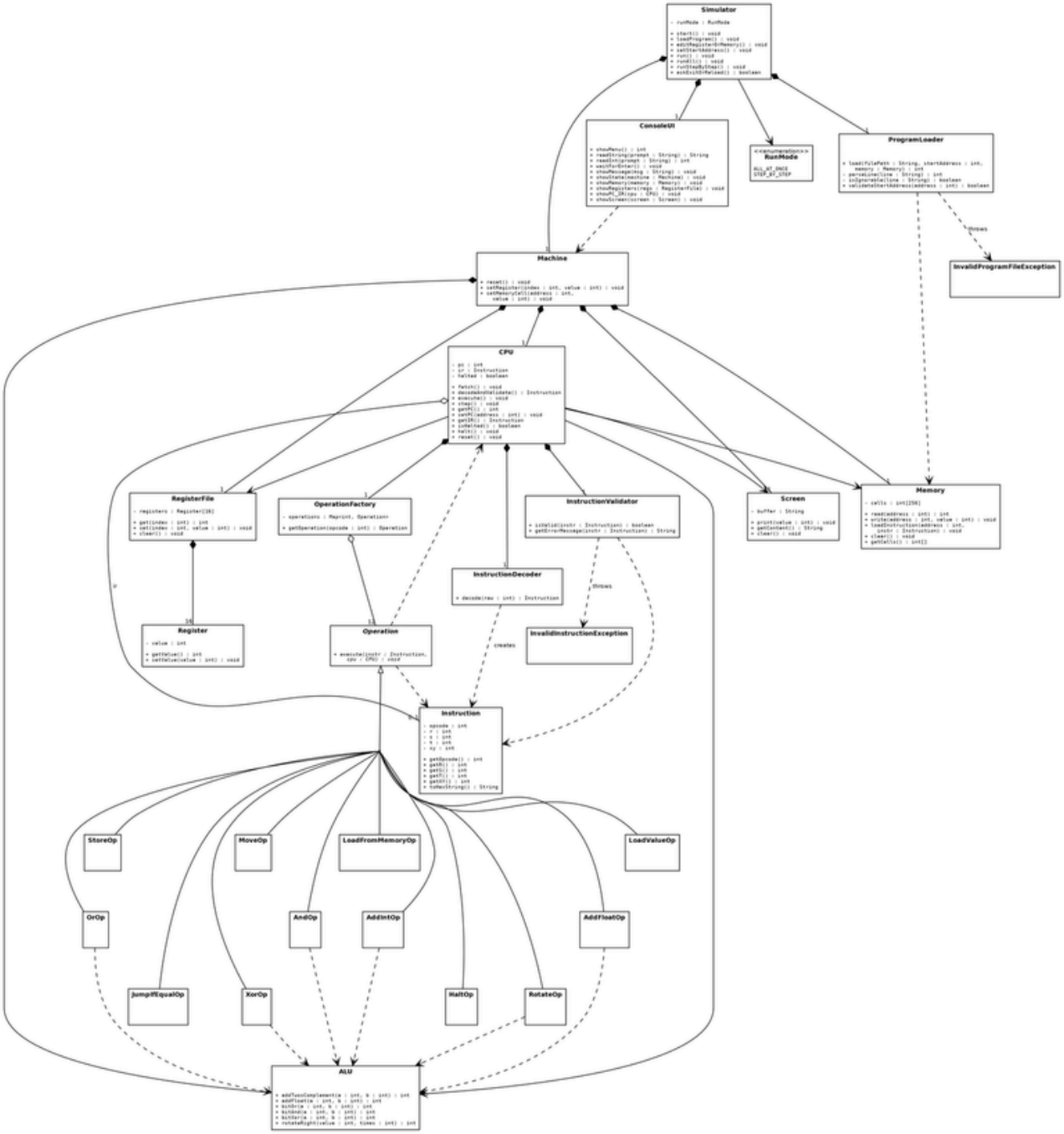

CPU --> Memory
CPU --> RegisterFile
CPU --> Screen
```

```
CPU --> ALU
Simulator *-- "1" Machine
Simulator *-- "1" ConsoleUI
Simulator *-- "1" ProgramLoader
Simulator --> RunMode
OperationFactory o-- "12" Operation


 Operation <|-- LoadFromMemoryOp
Operation    <|--     LoadValueOp
Operation      <|--       StoreOp
Operation <|-- MoveOp
Operation     <|--      AddIntOp
Operation    <|--     AddFloatOp
Operation  <|--  OrOp
Operation <|-- AndOp
Operation <|-- XorOp
Operation     <|--      RotateOp
Operation   <|--   JumpIfEqualOp
Operation <|-- HaltOp


InstructionDecoder      ..>      Instruction      :      creates
InstructionValidator ..> Instruction
Operation ..> Instruction
Operation ..> CPU
ProgramLoader ..> Memory
ProgramLoader    ..>    InvalidProgramFileException    :
throws
InstructionValidator ..> InvalidInstructionException : throws
ConsoleUI ..> Machine
AddIntOp ..> ALU
AddFloatOp ..> ALU
OrOp ..> ALU
AndOp ..> ALU
XorOp ..> ALU
RotateOp ..> ALU
@enduml
```

UML class diagram - Claude



4. Final Design

PlantUML code

```
@startuml
skinparamclassAttributeIconSize 0


class VoleSimulator {
 - runMode : RunMode
 + start() : void
 + loadProgram() : void
 + editRegister() : void
 + editMemoryCell() : void
 + setStartAddress() : void
+ chooseRunMode() : void
 + run() : void
 + runAll() : void
 + runStep() : void
 + askExitOrLoad() : boolean
}


enum RunMode {
ALL_AT_ONCE
STEP_BY_STEP
}

class ConsoleUI {
 + showMenu() : int
 + readFilePath() : String
 + readAddress() : int
 + readRegisterIndex() : int
 + readByteValue() : int
 + waitForEnter() : void
 + showMessage(msg : String) : void
 + showError(msg : String) : void
 + showMemoryMatrix(memory : Memory) : void +
showRegisters(regs : RegisterFile) : void +
showPCAndIR(cpu : CPU) : void
 + showScreen(screen : Screen) : void
 + showState(machine : Machine) : void
}


class ProgramLoader {
 + load(filePath : String, start : int, memory : Memory) : int
- readLines(filePath : String) : String[]
 - cleanLine(line : String) : String
 - isValidHexLine(s : String) : boolean
 + isValidStartAddress(address : int) : boolean
}

class Machine {
 - cpu : CPU
 - memory : Memory
 - registers : RegisterFile
 - screen : Screen
 - alu : ALU
 + loadProgram(words : String[], start : int) : void
+ setPC(address : int) : void
 + setRegister(index : int, value:int):void
 + setMemoryCell(address : int,value : int) : void +
step() : boolean
 + runAll() : void
 + isHalted() : boolean
 + reset() : void
 + getCPU() : CPU
 + getMemory() : Memory
 + getRegisters() : RegisterFile
+ getScreen() : Screen
 + getALU() : ALU
}


class CPU {
 - pc : int
 - ir : String
 - halted : boolean
 + fetch(memory : Memory) : void
 + decode() : Instruction
 + validate(instr : Instruction) : void
 + execute(instr : Instruction, machine : Machine) : void
```

```
 + step(machine : Machine) : void
+ getPC() : int
 + setPC(address : int) : void
 + getIR() : String
 + isHalted() : boolean
 + halt() : void
 + reset() : void
}


class Memory {
 - cells : int[256]
 + read(address : int) : int
 + write(address : int, value : int) : void
+ isValidAddress(address : int) : boolean
+ getCells() : int[]
 + clear() : void
}


class RegisterFile {
 - registers : int[16]
 + get(index : int) : int
 + set(index : int, value : int) : void
+ isValidIndex(index : int) : boolean
+ getAll() : int[]
 + clear() : void
}


class Screen {
 - buffer : String
 + print(value : int) : void
+ getContent() : String
 + clear() : void
}


class ALU {
 + addTwosComplement(a : int, b : int) : int
 + addFloat(a : int, b : int) : int
 + bitOr(a : int, b : int) : int
 + bitAnd(a : int, b : int) : int
 + bitXor(a : int, b : int) : int
 + rotateRight(value : int, times : int) : int }



class Instruction {
 - raw : String
 - opcode : int
 - r : int
 - s : int
 - t : int
 - x : int
 - y : int
 + getOpcode() : int
 + getR() : int
 + getS() : int
 + getT() : int
 + getXY() : int
 + toHexString() : String }



class InstructionDecoder {
 + decode(raw : String) : Instruction
}

class InstructionValidator {
 + isValid(instr : Instruction) : boolean
 + getErrorMessage(instr : Instruction) : String
}

abstract class Operation {
 + {abstract} execute(instr : Instruction, machine : Machine) : void
}

 class LoadFromMemoryOp
class       LoadValueOp
class    StoreOp
class     MoveOp
class   AddIntOp
class AddFloatOp
class OrOp
class      AndOp
class      XorOp
class RotateOp
class    JumpIfEqualOp
class HaltOp


class OperationFactory {
 - operations : Map<int, Operation>
 + getOperation(opcode : int) : Operation
}

class InvalidInstructionException
class InvalidProgramFileException

VoleSimulator *-- "1" Machine
VoleSimulator *-- "1" ConsoleUI
VoleSimulator *-- "1" ProgramLoader
VoleSimulator --> RunMode

Machine *-- "1" CPU
Machine *-- "1" Memory
Machine *-- "1" RegisterFile
```

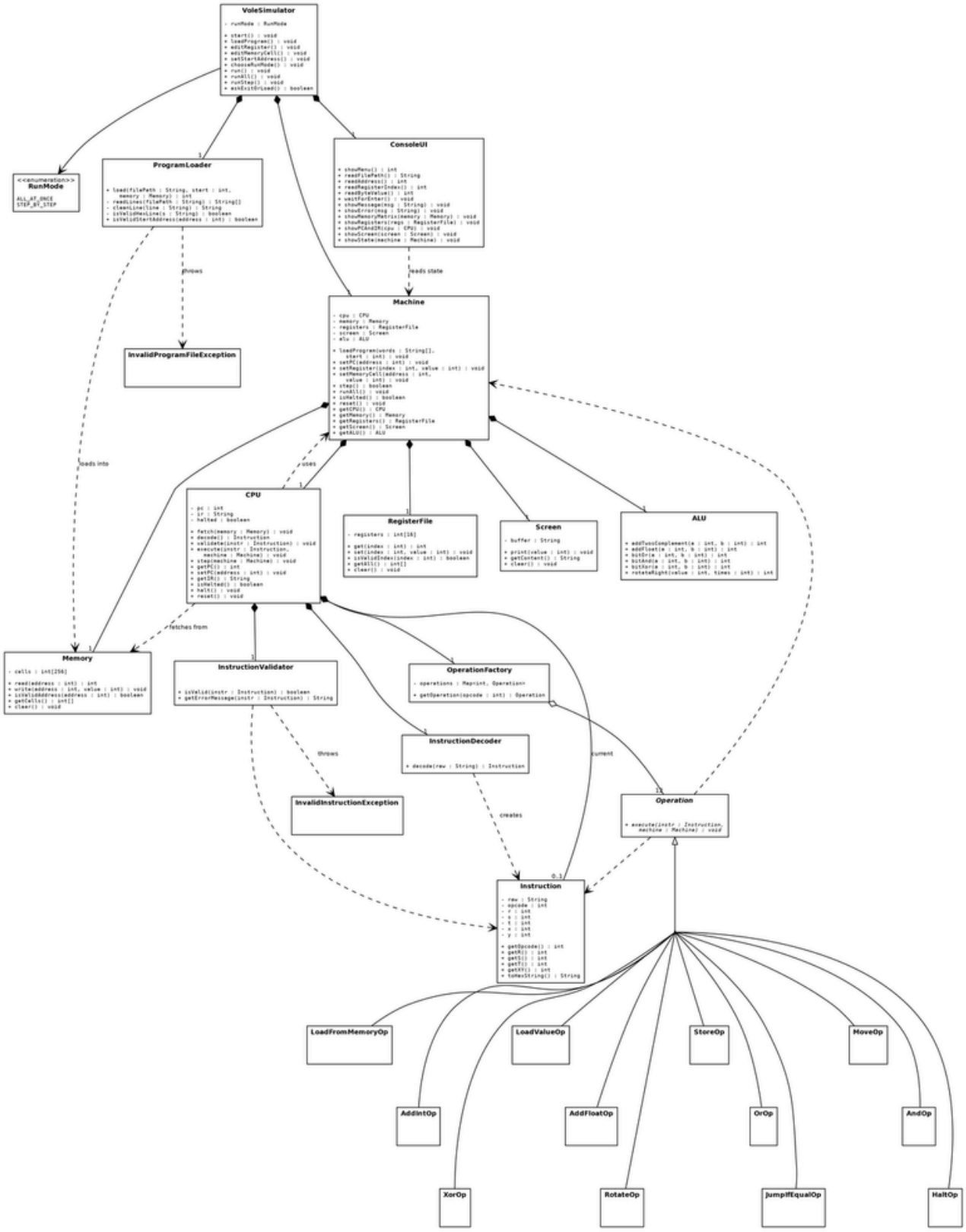
```
Machine *-- "1" Screen
Machine *-- "1" ALU

CPU *-- "1" InstructionDecoder
CPU *-- "1" InstructionValidator
CPU *-- "1" OperationFactory
CPU o-- "0..1" Instruction : current
CPU ..> Memory : fetches from
CPU ..> Machine : uses


 OperationFactory o-- "12" Operation
Operation    <|--    LoadFromMemoryOp
Operation       <|--       LoadValueOp
Operation   <|--   StoreOp
Operation    <|--   MoveOp
Operation  <|--  AddIntOp
Operation <|-- AddFloatOp
Operation    <|--    OrOp
Operation    <|--    AndOp
Operation    <|--    XorOp
Operation <|-- RotateOp
Operation     <|--     JumpIfEqualOp
Operation <|-- HaltOp


Operation ..> Instruction
Operation ..> Machine
InstructionDecoder ..> Instruction : creates
InstructionValidator ..> Instruction
InstructionValidator ..> InvalidInstructionException : throws
ProgramLoader ..> Memory : loads into
ProgramLoader ..> InvalidProgramFileException : throws
ConsoleUI ..> Machine : reads state
@enduml
```

UML class diagram - Final design



Why it is the best design

- Each classhas one responsibility: hardware (Machine, CPU, Memory, RegisterFile, Screen, ALU), instruction handling (decoder, validator, operations) and user interface (VoleSimulator, ConsoleUI, ProgramLoader).
- One Operation class per opcode and an OperationFactory remove the large switch from the CPU, so a new instruction needs only a new class.
- The decode, validate, execute cycle in CPU.step() matches the required fetch, validate, decode, execute, update PC.
- The UI never touches the hardware logic directly, so the machine can be tested without the console.
- Invalid instructionsand invalid program filesarerejected through dedicated exceptions.

Experience and what we learned

1-AI toolsgive good design ideas fast,buteach has weaknesses, so we must review and compare their output instead of accepting it as it is.
2- Combining the best parts of several designs gives a better result than relying on a single one, because each design solved a different part of the problem well.

5. Two Vole programs

Program 1: perimeter of a rectangle

```
; ------------------------------------------------
; Vole Machine Program 1
; Purpose: Calculate the perimeter of a rectangle
; (width = 5, height = 3) and display it: 2*(5+3) = 16 = 10h ;
------------------------------------------------
2105 ; R1 = 05 (width)
2203 ; R2 = 03 (height)
5312 ; R3 = R1 + R2
5433 ; R4 = R3 + R3 (perimeter)
34FF ; Store R4 to FF (screen)
C000 ; Halt
```

Program 2: sum of 1 to 5 using a loop

```
; ------------------------------------------------
; Vole Machine Program 2
; Purpose: Calculate 1+2+3+4+5 using a loop and display
; the result: 15 = 0F. Change the first line to change N.
; NOTE: load this program at address 00 (jumps are absolute).
; ------------------------------------------------
2005 ; 00: R0 = 05 (N, the limit)
2100 ; 02: R1 = 00 (counter i)
2200 ; 04: R2 = 00 (sum)
2301 ; 06: R3 = 01 (constant one)
5113 ; 08: R1 = R1 + R3 (i = i + 1)
5221 ; 0A: R2 = R2 + R1 (sum = sum + i)
B110 ; 0C: if R1 == R0 jump to 10 (end of loop)
B008 ; 0E: R0 == R0 always, so jump back to 08
32FF ; 10: Store R2 to FF (screen)
C000 ; 12: Halt
```

6. Video Link

https://drive.google.com/file/d/1FLpjH7XvfoSpElwnhHI7PT6Vu3nYJMu5/view?usp=drivesdk